EXPERIMENT – 7

HANDLING OUTLIERS & MISSING VALUES IN A DATASET

AIM:

To preprocess the dataset by removing duplicates, detecting outliers, and handling missing values using Pandas and Scikit-learn SimpleImputer

PREREQUISITES & REQUIREMENTS:

1. Computer with Python Installed
2. A dataset for Practice
3. Jupyter Notebook
4. Knowledge on Pandas & Scikit-Learn Libraries

Step 1: Import the basic libraries

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load & Analyze the Dataset

In [ ]:
```
# Load the employee dataset
df_241UDS0096 = pd.read_excel("/content/employee dataset.xlsx")

# Dataset Information
df_241UDS0096.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 223 entries, 0 to 222
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Department          221 non-null    object 
 1   Age                 221 non-null    float64
 2   Experience_Years    221 non-null    float64
 3   Monthly_Salary      221 non-null    float64
 4   Performance_Score   221 non-null    float64
 5   Projects_Completed  223 non-null    int64  
 6   Training_Hours      223 non-null    int64  
 7   Overtime_Hours      223 non-null    int64  
 8   Satisfaction_Score  223 non-null    float64
dtypes: float64(5), int64(3), object(1)
memory usage: 15.8+ KB
```

In [ ]:
```
#Display the first five records
df_241UDS0096.head()
```

	Department	Age	Experience_Years	Monthly_Salary	Performance_Score	Projects_Completed	Training_Hours	Overtime_Hours	Satisfaction_Score
0	Finance	37.0	11.8	75682.0	87.4	10	70	21	7.0
1	R&D	21.0	4.3	81494.0	94.2	5	59	25	7.1
2	Operations	36.0	10.8	103317.0	51.3	1	46	17	7.9
3	Operations	32.0	10.0	87215.0	82.3	15	38	13	3.1
4	IT	47.0	15.4	100587.0	92.9	4	34	28	7.7

In [ ]:
```
#Check the shape
df_241UDS0096.shape
```

```
(223, 9)
```

In [ ]:
```
# Remove completely empty rows
df_241UDS0096 = df_241UDS0096.dropna(how='all')

df_241UDS0096.shape
```

```
(223, 9)
```

Step 3: Find & Remove the Duplicates

In [ ]:
```
# Checking the number of duplicates
df_241UDS0096[
    ['Age', 'Experience_Years', 'Performance_Score']
].duplicated().sum()
```

```
np.int64(3)
```

In [ ]:
```
# Remove the duplicates
df_241UDS0096 = df_241UDS0096.drop_duplicates()

df_241UDS0096.shape
```

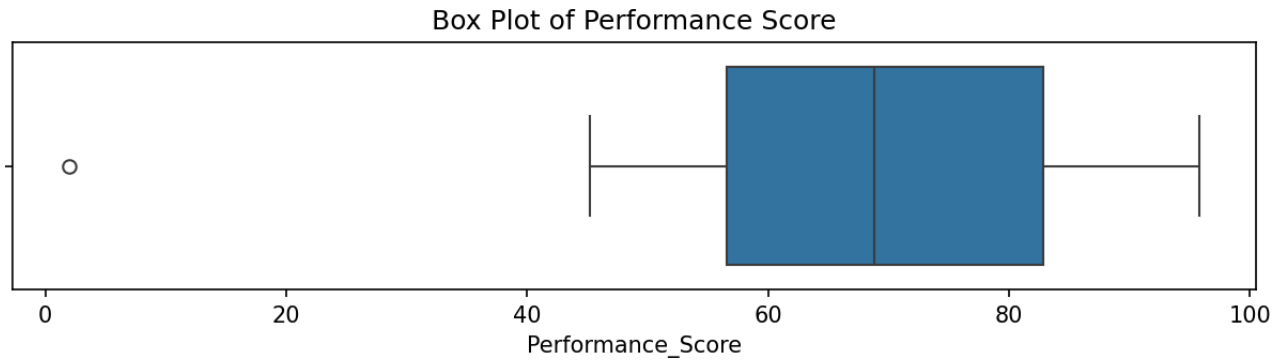
```
(220, 9)
```

Step 4: Identify & Remove the Outliers

In [ ]:
```
plt.figure(figsize=(10, 2), dpi=150)

sns.boxplot(
    data=df_241UDS0096['Performance_Score'],
    orient='h'
)

plt.title('Box Plot of Performance Score')
plt.show()
```

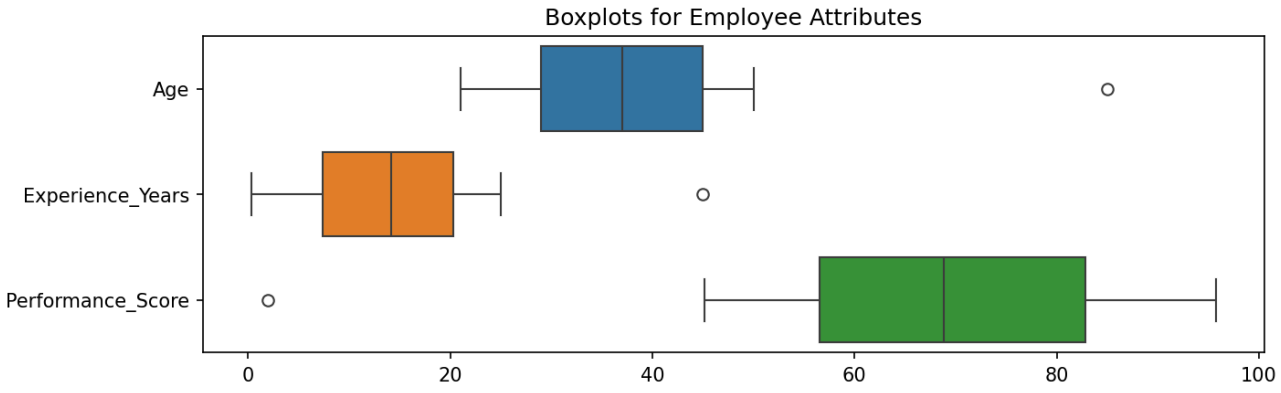


In [ ]:
```
#Create a list for outlier columns
columns_241UDS0096 = [
    'Age',
    'Experience_Years',
    'Performance_Score'
]
```

In [ ]:
```
#Visualization with Boxplots
plt.figure(figsize=(10, 3), dpi=150)

sns.boxplot(
    data=df_241UDS0096[columns_241UDS0096],
    orient='h'
)

plt.title('Boxplots for Employee Attributes')
plt.show()
```



In [ ]:
```
#Calculate the IQR Ranges Q1 and Q3
Q1_241UDS0096 = df_241UDS0096[
    columns_241UDS0096
].quantile(0.25)

Q3_241UDS0096 = df_241UDS0096[
    columns_241UDS0096
].quantile(0.75)

Q1_241UDS0096, Q3_241UDS0096
```

```
(Age                 29.00
 Experience_Years     7.40
 Performance_Score   56.55
 Name: 0.25, dtype: float64,
 Age                 45.000
 Experience_Years    20.300
 Performance_Score   82.875
 Name: 0.75, dtype: float64)
```

In [ ]:
```
#Calculate IQR
IQR_241UDS0096 = Q3_241UDS0096 - Q1_241UDS0096

IQR_241UDS0096
```

	0
Age	16.000
Experience_Years	12.900
Performance_Score	26.325

dtype: float64

In [ ]:
```
#Find the Outliers using IQR Range
outliers_241UDS0096 = (
    (df_241UDS0096[columns_241UDS0096] <
     (Q1_241UDS0096 - 1.5 * IQR_241UDS0096))
    |
    (df_241UDS0096[columns_241UDS0096] >
     (Q3_241UDS0096 + 1.5 * IQR_241UDS0096))
)
```

In [ ]:
```
outliers_241UDS0096.sum()
```

	0
Age	1
Experience_Years	1
Performance_Score	1

dtype: int64

In [ ]:
```
df_241UDS0096.shape
```

```
(220, 9)
```

In [ ]:
```
# Removing the outliers
df_241UDS0096 = df_241UDS0096[
    ~(outliers_241UDS0096).any(axis=1)
]
```

In [ ]:
```
df_241UDS0096.head()
```

	Department	Age	Experience_Years	Monthly_Salary	Performance_Score	Projects_Completed	Training_Hours	Overtime_Hours	Satisfaction_Score
0	Finance	37.0	11.8	75682.0	87.4	10	70	21	7.0
1	R&D	21.0	4.3	81494.0	94.2	5	59	25	7.1
2	Operations	36.0	10.8	103317.0	51.3	1	46	17	7.9
3	Operations	32.0	10.0	87215.0	82.3	15	38	13	3.1
4	IT	47.0	15.4	100587.0	92.9	4	34	28	7.7

Step 5: Fill the Missing Values

In [ ]:
```
# Find any missing values
df_241UDS0096.isnull().sum()
```

	0
Department	2
Age	2
Experience_Years	2
Monthly_Salary	2
Performance_Score	2
Projects_Completed	0
Training_Hours	0
Overtime_Hours	0
Satisfaction_Score	0

dtype: int64

In [ ]:
```
# Filling Age with mean value
df_241UDS0096['Age'] = df_241UDS0096[
    'Age'
].fillna(
    np.mean(df_241UDS0096['Age'])
)
```

In [ ]:
```
df_241UDS0096['Age'].isnull().sum()
```

```
np.int64(0)
```

In [ ]:
```
df_241UDS0096['Age'].isnull().sum()
```

```
np.int64(0)
```

In [38]:
```
#Import SimpleImputer
from sklearn.impute import SimpleImputer
```

In [ ]:
```
#Create the Imputer
numerical_imputer_241UDS0096 = SimpleImputer(
    strategy='median'
)
```

In [ ]:
```
#Apply Median Imputation
df_241UDS0096[
    ['Experience_Years', 'Performance_Score']
] = numerical_imputer_241UDS0096.fit_transform(
    df_241UDS0096[['Experience_Years', 'Performance_Score']]
)
```

In [ ]:
```
#Check Missing Values
df_241UDS0096[
    ['Experience_Years', 'Performance_Score']
].isnull().sum()
```

	0
Experience_Years	0
Performance_Score	0

dtype: int64

Filling Department with Mode using SimpleImputer

In [ ]:
```
#First check the Department frequencies:

df_241UDS0096['Department'].value_counts()
```

Department	count
IT	51
Operations	39
HR	36
Sales	34
Marketing	21
R&D	19
Finance	15

dtype: int64

Create Categorical Imputer

In [ ]:
```
categorical_imputer_241UDS0096 = SimpleImputer(
    strategy='most_frequent'
)
```

Impute Missing Department Values

In [ ]:
```
df_241UDS0096[['Department']] = categorical_imputer_241UDS0096.fit_transform(
    df_241UDS0096[['Department']]
)
```

Check the Department Values

In [ ]:
```
df_241UDS0096['Department'].value_counts()
```

	count
Department	
IT	53
Operations	39
HR	36
Sales	34
Marketing	21
R&D	19
Finance	15

dtype: int64

In [ ]:
```
# Cleaned Dataset Information
df_241UDS0096.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 217 entries, 0 to 219
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Department          217 non-null    object 
 1   Age                 217 non-null    float64
 2   Experience_Years    217 non-null    float64
 3   Monthly_Salary      215 non-null    float64
 4   Performance_Score   217 non-null    float64
 5   Projects_Completed  217 non-null    int64  
 6   Training_Hours      217 non-null    int64  
 7   Overtime_Hours      217 non-null    int64  
 8   Satisfaction_Score  217 non-null    float64
dtypes: float64(5), int64(3), object(1)
memory usage: 17.0+ KB
```

Check all remaining missing values

In [ ]:
```
df_241UDS0096.isnull().sum()
```

	0
Department	0
Age	0
Experience_Years	0
Monthly_Salary	2
Performance_Score	0
Projects_Completed	0
Training_Hours	0
Overtime_Hours	0
Satisfaction_Score	0

dtype: int64

RESULT:

By using Pandas and Scikit-learn, duplicates were removed, outliers were detected and dropped using the IQR method, and missing values in the employee performance dataset were filled using mean, median, and most frequent methods. The dataset is now clean and ready for further analysis.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.